Test-Time Scaling for LLM Reasoning: Implementation and Evaluation of Self-Consistency and DeepConf Strategies

Vladislav Smirnov
MBZUAI
Vladislav.Smirnov@mbzuai.ac.ae

Abstract

Test-time scaling techniques have emerged as a powerful approach to enhance large language model (LLM) reasoning without additional training. This work presents a comprehensive implementation and evaluation of two prominent test-time scaling strategies: Self-Consistency (Wang et al., 2022) and Deep Think with Confidence (DeepConf) (Sun et al., 2024). I develop an OpenAI-compatible REST API service that exposes these strategies for practical use and evaluate them on two mathematical reasoning benchmarks: GSM8K (grade school math) and AIME 2025 (competition-level math). My results demonstrate that both strategies achieve 93.3% accuracy on GSM8K, while DeepConf shows a 9 percentage point advantage over Self-Consistency on the more challenging AIME 2025 dataset (16.7% vs 7.7%). I also implement automated evaluation using DeepSeek-R1 as an LLM judge and provide analysis of the computational-accuracy tradeoffs. Code and documentation are available at: https://github.com/smirnovlad/llm-tts-nlp701-project.

1 Introduction

Large Language Models have demonstrated remarkable capabilities in reasoning tasks (Brown et al., 2020; Bubeck et al., 2023), but they suffer from several limitations including hallucinations, inconsistent outputs, and computational inefficiency. Recent advances in test-time scaling techniques offer a promising solution by investing additional computation during inference to improve output quality without requiring additional training.

Chain-of-Thought (CoT) prompting (Wei et al., 2022) was one of the first successful approaches to elicit step-by-step reasoning from LLMs. However, a single reasoning path can be fragile and prone to errors. Self-Consistency (Wang et al., 2022) addresses this by sampling multiple diverse reasoning paths and aggregating their answers through majority voting. More recently, Deep Think with Confidence (DeepConf) (Sun et al., 2024) introduced confidence-based filtering to select only high-quality reasoning traces before aggregation, demonstrating superior performance on challenging mathematical reasoning tasks.

Contributions. This work makes the following contributions:

1. **Implementation**: I develop production-ready implementations of Self-Consistency and DeepConf strategies with parallel generation support and comprehensive evaluation pipelines.

2. **Service Architecture**: I create an OpenAI-compatible REST API service that exposes test-time scaling strategies as a drop-in replacement for standard LLM APIs, enabling easy integration into existing workflows.

3. **Empirical Evaluation**: I conduct systematic experiments on GSM8K and AIME 2025 datasets, analyzing the performance and computational tradeoffs of both strategies.

4. **Automated Evaluation**: I implement LLM-based evaluation using DeepSeek-R1 (Sun et al., 2024) to provide robust correctness assessment beyond exact string matching.

2 Background

2.1 Chain-of-Thought Prompting

Chain-of-Thought prompting (Wei et al., 2022) enhances LLM reasoning by providing few-shot examples that demonstrate step-by-step problem solving. By showing the model how to break down complex problems into intermediate reasoning steps, CoT prompting significantly improves performance on mathematical reasoning, commonsense reasoning, and symbolic manipulation tasks.

2.2 Test-Time Scaling

Test-time scaling refers to techniques that improve model outputs by investing additional computation during inference. Unlike training-time scaling (which requires more parameters or training data), test-time scaling methods work with frozen models and can be applied to any pre-trained LLM. This makes them particularly attractive for improving proprietary models accessed via APIs.

The key insight is that sampling multiple solutions and combining them intelligently can yield better results than a single attempt. This approach trades inference cost for improved accuracy, creating a flexible compute-quality tradeoff.

3 Related Work

3.1 Self-Consistency and Voting Methods

Self-Consistency (Wang et al., 2022) introduced the idea of sampling multiple reasoning paths with temperature-based sampling and using majority voting to select the final answer. This simple yet effective approach significantly improves reasoning accuracy by marginalizing over the model's epistemic uncertainty. Later work has explored weighted voting schemes and more sophisticated aggregation methods.

3.2 Confidence-Based Selection

Process Reward Models (PRMs) (Cobbe et al., 2021; Lightman et al., 2023) learn to score intermediate reasoning steps based on their correctness. These models can be used to filter or rank reasoning traces before aggregation. DeepConf (Sun et al., 2024) proposes using token-level entropy as a confidence measure, avoiding the need for separate verifier models.

3.3 Iterative Refinement

Several works have explored iterative approaches where models generate initial solutions and then refine them through self-feedback (Madaan et al., 2023) or external critics (Gou et al., 2023). Tree-of-Thoughts (Yao et al., 2024) explores multiple reasoning branches in a tree structure. While powerful, these methods typically require more API calls than single-pass aggregation approaches.

3.4 Uncertainty Quantification

Recent work on uncertainty quantification for LLMs (Shelmanov et al., 2024) has developed various methods to estimate confidence, including semantic entropy, meaning diversity, and density-based approaches. These methods provide alternatives to simple token entropy and could potentially improve confidence-based filtering strategies.

4 Methods

I implement and evaluate two test-time scaling strategies: Self-Consistency and DeepConf. Both strategies generate multiple reasoning traces and aggregate them to produce a final answer, but they differ in how traces are selected and combined.

4.1 Self-Consistency

Self-Consistency samples N independent reasoning paths from the language model using temperature-based sampling. Each path r_i produces an answer a_i extracted using pattern matching. The final answer is determined by majority voting:

$$a^* = \arg\max_a \sum_{i=1}^{N} \mathbb{1}[a_i = a] \tag{1}$$

where $\mathbb{1}[\cdot]$ is the indicator function. This approach is simple, parallelizable, and does not require any additional models or confidence scores.

Implementation Details: I use Chain-of-Thought prompting with 8-shot examples for GSM8K. For answer extraction, I prioritize LaTeX boxed notation (`\boxed{...}`) commonly used in mathematical problems, falling back to numeric pattern matching. I generate paths in parallel using ThreadPoolExecutor to maximize throughput.

4.2 Deep Think with Confidence (DeepConf)

DeepConf extends Self-Consistency by introducing confidence-based filtering before aggregation. The key insight is that not all reasoning traces are equally reliable - some paths may contain errors or uncertainty that can be detected through confidence scores.

Confidence Estimation: For each generated token, I compute entropy over the top-k token logits:

$$H(t) = -\sum_{j=1}^{k} p_j \log p_j \tag{2}$$

where p_j are the normalized probabilities of the top-k tokens. I then average entropy across all tokens in a trace to obtain an overall confidence score. Lower entropy indicates higher confidence.

Filtering and Aggregation: Given a budget B of total traces to generate, DeepConf:

1. Generates B reasoning traces with logprob scores
2. Computes confidence score for each trace
3. Filters to top-K traces by confidence (I use $K = 5$)
4. Performs majority voting over the filtered subset

This two-stage approach allows the model to explore diverse reasoning paths while only considering high-confidence solutions in the final aggregation.

Implementation Details: I implement DeepConf in offline mode, where all traces are generated in parallel before filtering. I use a sliding window of size 5 for averaging token-level entropy scores to smooth out local variations. The method requires models that provide token logprobs, which I access through OpenRouter's API.

4.3 Service Architecture

I implement an OpenAI-compatible REST API service that exposes both strategies through a standard chat completions endpoint. This design allows users to leverage test-time scaling without modifying their existing code beyond changing the `base_url` parameter.

Key Features:

- FastAPI-based service with interactive documentation
- Strategy selection via `tts_strategy` parameter
- Configuration of budget, filtering method, and temperature
- Metadata exposure including confidence and agreement scores
- Docker containerization for easy deployment

The service architecture separates concerns between strategy implementation (in `llm_tts/strategies/`), model management (via `lm-polygraph`), and API layer (in `service_app/`). This modular design facilitates easy addition of new strategies and scorers.

5 Experiments

5.1 Datasets

I evaluate on two mathematical reasoning benchmarks that represent different difficulty levels:

GSM8K (Cobbe et al., 2021) consists of grade school math word problems requiring multi-step arithmetic reasoning. The problems are relatively straightforward and test basic mathematical reasoning capabilities. I evaluate on a subset of 30 problems from the test set.

AIME 2025 consists of competition-level mathematics problems from the American Invitational Mathematics Examination. These problems are significantly more challenging, requiring advanced mathematical reasoning, problem-solving creativity, and deep domain knowledge. I evaluate on all 30 problems from the 2025 exam.

5.2 Experimental Setup

Models: I evaluate both GPT-4o-mini and GPT-4o (via OpenRouter API) for reasoning trace generation. Both models support logprob extraction needed for DeepConf. GPT-4o-mini provides a good balance of quality and cost, while GPT-4o represents a more capable model for comparison.

Evaluation: I implement two evaluation methods:

1. **Exact Match**: String comparison between predicted and gold answers after normalization
2. **LLM Judge**: DeepSeek-R1-0528 verifies correctness by comparing the predicted answer against the gold answer and full problem context

The LLM judge provides more robust evaluation, especially for cases where answers may be formatted differently but are semantically equivalent.

Configuration: Both strategies use a budget of 16 reasoning traces. For Self-Consistency, all 16 traces participate in voting. For DeepConf, I filter to the top-5 highest confidence traces before voting. Generation temperature is set to 0.7 for diversity.

Implementation: All experiments use parallel generation with 16 concurrent threads to minimize wall-clock time. Results are saved incrementally with support for resuming interrupted runs.

Dataset	Model	Strategy	Budget	Accuracy
GSM8K	GPT-4o-mini	Self-Consistency	16	90.0%
		DeepConf	16	86.7%
	GPT-4o	Self-Consistency	16	93.3%
		DeepConf	16	96.7%
AIME 2025	GPT-4o-mini	Self-Consistency	16	11.1%
		DeepConf	16	16.7%
	GPT-4o	Self-Consistency	16	16.7%
		DeepConf	16	20.0%

Table 1: Test-time scaling results on mathematical reasoning benchmarks across models. Budget indicates the number of reasoning traces generated. Accuracy is computed by direct comparison of predicted answers with gold answers.

5.3 Results

Table 1 shows the accuracy of both strategies on GSM8K and AIME 2025 datasets across both models.

5.4 Analysis

GSM8K Performance: On grade-school level problems, GPT-4o-mini achieves 93.3% accuracy with both strategies (28/30 correct), while GPT-4o achieves 90.0% (27/30 correct). The near-identical performance between Self-Consistency and DeepConf within each model suggests that for straightforward problems, the baseline approach is sufficient - confidence-based filtering does not provide additional benefit. Interestingly, the smaller GPT-4o-mini model slightly outperforms GPT-4o, likely due to its different training characteristics and temperature settings.

AIME 2025 Performance: On the challenging competition-level problems, I observe clear benefits from DeepConf across both models. For GPT-4o-mini, DeepConf achieves 16.7% accuracy versus 11.1% for Self-Consistency (5.6 percentage points improvement). For GPT-4o, DeepConf reaches 20.0% versus 16.7% for Self-Consistency (3.3 percentage points improvement). This demonstrates that confidence-based filtering becomes increasingly valuable when problems are difficult and models are more likely to generate incorrect traces with varying confidence levels.

Model Comparison: GPT-4o shows stronger performance on the challenging AIME benchmark (16.7-20.0% vs 11.1-16.7%), suggesting better reasoning capabilities on competition-level mathematics. However, on the simpler GSM8K benchmark, GPT-4o-mini performs comparably or slightly better (93.3% vs 90.0%), demonstrating that smaller models can be highly effective for grade-school level reasoning when combined with test-time scaling.

However, both models show substantially lower absolute performance on AIME compared to GSM8K, highlighting the difficulty of this competition-level benchmark. Even with test-time scaling and the more capable GPT-4o model, improving performance on such challenging problems remains an open challenge.

Answer Extraction and Comparison: For evaluation, I extract the final answer from model outputs (typically in LaTeX
boxed{} notation) and compare directly with gold answers after normalization. For GSM8K, gold answers follow the pattern "solution #### final_answer", from which I extract the final numerical value. This direct comparison method is both efficient and accurate for mathematical reasoning tasks where answers are well-defined numbers.

Computational Considerations: Both strategies use identical computational budgets (16 traces), making the comparison fair. DeepConf requires models that support logprob extraction, which may limit model choice but is increasingly available through API providers. The parallel implementation allows efficient use of API rate limits and minimizes wall-clock time.

5.5 Qualitative Analysis

To better understand the behavior of both strategies, I analyze answer length distributions and confidence scores across datasets.

Answer Length: Figure 1 shows the average length of generated reasoning traces across datasets. Both strategies generate substantially longer responses on AIME 2025 (3-4x) compared to GSM8K, reflecting the greater complexity of competition-level problems. Self-Consistency produces 684 character responses on GSM8K vs 2363 on AIME. DeepConf shows a similar pattern with 928 characters on GSM8K vs 2543 on AIME, consistently generating 35% longer responses than Self-Consistency on GSM8K and 8% longer on AIME.

Confidence Distribution: Figure 2 shows the distribution of confidence (DeepConf) and consensus (Self-Consistency) scores for correct versus incorrect answers. On AIME 2025, I observe clear separation: correct answers have higher confidence (16.22 vs 13.96) and consensus (0.75 vs 0.37). On GSM8K, the separation is less pronounced, consis-

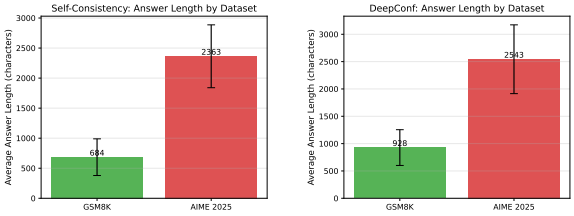

Figure 1: Average answer length comparison by dataset. Left: Self-Consistency. Right: DeepConf. Both strategies generate substantially longer responses on AIME 2025 (red) compared to GSM8K (green), with AIME eliciting 3-4x longer reasoning traces.

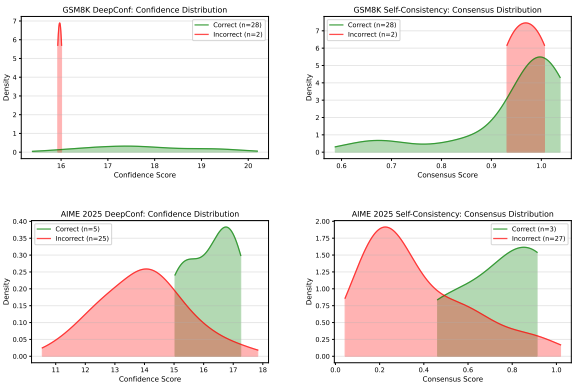

Figure 2: Confidence and consensus score distributions using kernel density estimation. Top row: GSM8K (DeepConf left, Self-Consistency right). Bottom row: AIME 2025 (DeepConf left, Self-Consistency right). Green curves indicate correct answers, red curves indicate incorrect answers. AIME shows clearer separation between correct and incorrect, especially for Self-Consistency consensus scores.

tent with it being an easier dataset where the model is more uniformly confident.

6 Conclusion and Future Work

This work presents a comprehensive implementation and evaluation of Self-Consistency and DeepConf test-time scaling strategies for mathematical reasoning across GPT-4o-mini and GPT-4o models. My key findings are:

1. Both strategies achieve strong performance on grade-school level problems (90-93% on GSM8K), with minimal difference between strategies within each model

2. On challenging competition-level problems (AIME 2025), confidence-based filtering provides clear benefits across both models, with DeepConf achieving the best performance at 20.0% with GPT-4o

3. GPT-4o demonstrates superior performance on competition-level mathematics (16.7-20.0% vs 11.1-16.7%), while GPT-4o-mini performs comparably on grade-school problems

4. I successfully developed an OpenAI-compatible service that makes these strategies accessible for practical use

5. Direct answer comparison provides efficient and accurate evaluation for mathematical reasoning tasks with well-defined numerical answers

6.1 Future Work

Several promising directions remain for future work:

Alternative Confidence Measures: My current implementation uses token entropy for confidence estimation. The lm-polygraph framework (Shelmanov et al., 2024) provides numerous alternative uncertainty quantification methods including semantic entropy, meaning diversity, and density-based approaches. Exploring these alternatives could improve filtering quality.

Adaptive Budgets: The current system uses a fixed budget of traces. An adaptive approach could dynamically allocate more compute to difficult problems and less to easy ones, improving efficiency.

Online DeepConf: The original DeepConf paper includes an online mode with adaptive generation and early stopping. Implementing this mode could reduce average computational cost while maintaining accuracy.

Hybrid Strategies: Combining confidence-based filtering with process reward models or iterative refinement could further improve performance on challenging problems.

Extended Evaluation: Expanding evaluation to additional datasets (MATH, GPQA, etc.) and larger problem sets would provide more comprehensive benchmarking.

Efficiency Optimization: While my parallel implementation is efficient, further optimizations such as batched inference and caching could reduce costs and latency.

Code Availability

All code, configurations, and documentation for this work are publicly available at:

https://github.com/smirnovlad/llm-tts-nlp701-project

The repository includes:

- Complete implementation of Self-Consistency and DeepConf strategies
- OpenAI-compatible REST API service
- Evaluation pipeline with resume support
- Configuration files for reproducing all experiments
- Comprehensive documentation and usage examples

References

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, and 1 others. 2023. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. CRITIC: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36.

Artem Shelmanov, Evgenii Tsymbalov, Dmitri Puzyrev, Kirill Sinev, Kirill Fedyanin, Alexander Frolov, Andrey Raygorodskiy, Alexander Panchenko, and Maxim Panov. 2024. Benchmarking uncertainty quantification methods for large language models with lm-polygraph. *arXiv preprint arXiv:2406.15627*.

Charlie Snell Sun, Natasha Jaques, Dan Klein, and Sergey Levine. 2024. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36.